Go2 Hopping Setup and Experiment Plan

1 Current Task

The current task is hopping control for Go2. At reset, each environment samples a scalar target apex-height command. The full command range is

$$h^* \sim \mathcal{U}(0.5, 1.2) \text{ m}, \tag{1}$$

but during the initial hopping-discovery curriculum it is fixed to $h^* = 0.5$ m. The initial drop height is sampled independently throughout training:

$$h_{\text{drop}} \sim \mathcal{U}(0.5, 1.2) \text{ m}. \tag{2}$$

The robot base is placed at h_{drop} with zero root velocity and the default joint state. The policy must keep hopping for the full episode. The objective is not to jump higher forever, nor merely to recover to the initial release height, but to make each rebound apex return close to the commanded target height while maintaining a flat quadruped posture and staying in place.

The episode continues until either a failure condition is met or the 20 s time limit is reached.

2 Environment Variants

The main environment IDs are:

ID	Notes
`Go2-Rebounce-Flat`	rigid plane, 4096 envs, env spacing 2.5 m
`Go2-Rebounce-Trampoline-Baseline`	instantaneous deployable actor
`Go2-Rebounce-Trampoline-Baseline-Base`	instantaneous actor with root pose and base linear velocity
`Go2-Rebounce-Trampoline-history`	flattened actor history, $H = 10$
`Go2-Rebounce-Trampoline-RNN`	recurrent actor, no flattened history
`Go2-Rebounce-Trampoline-Teacher`	privileged actor upper bound
`Go2-Rebounce-Trampoline-Student`	MLP student distilled from privileged teacher
`Go2-Rebounce-Trampoline-RNN-Student`	recurrent student distilled from privileged teacher

The trampoline variant keeps the same rebounce MDP terms but replaces the support with a deformable trampoline and repins the trampoline rim. Task success does not rely on rigid-deformable contact sensors. Apex and foot-clearance gates use only root state and kinematic foot body positions.

3 Timing

The simulator time step and action decimation are

$$\Delta t_{\text{sim}} = 0.005 \text{ s}, \qquad d = 4, \qquad \Delta t = d\Delta t_{\text{sim}} = 0.02 \text{ s}. \tag{3}$$

The current training episode length is

$$T_{\text{episode}} = 20 \text{ s}. \tag{4}$$

IsaacLab multiplies every weighted reward term by Δt, and logged `Episode_Reward/*` values are additionally divided by T_{episode}.

4 Reset and Command

The rebounce command is

$$\boldsymbol{c} = [h^*]. \tag{5}$$

In the current setup, the target rebound-apex height and the initial drop height are independent. During the first 1000 PPO iterations the target is fixed to 0.5 m; after that it uses the full range:

$$h^* = \begin{cases} 0.5, & N \le 1000 \times 24, \\ \mathcal{U}(0.5, 1.2), & N > 1000 \times 24, \end{cases} \qquad h_{\text{drop}} \sim \mathcal{U}(0.5, 1.2). \tag{6}$$

The reset event writes h^* into the command buffer and sets the root state to

$$\boldsymbol{p}_{\text{root}} = \boldsymbol{o}_{\text{env}} + \left[p_x^0,\ p_y^0,\ h_{\text{drop}} + h_{\text{offset}}\right], \qquad \boldsymbol{v}_{\text{root}} = \mathbf{0}, \tag{7}$$

where $h_{\text{offset}} = 0$ in the current config. Joint positions are reset to the default Go2 pose and joint velocities are reset to zero.

During the rollout, the target command is resampled after

$$t_{\text{resample}} \sim \mathcal{U}(10, 20) \text{ s}. \tag{8}$$

This changes only h^*; it does not teleport the robot or change the latched initial drop height h_0. With a 20 s episode, most rollouts see zero or one mid-episode target change, which is enough to test adaptation without turning the task into rapid command following.

The command term stores the following buffers:

Symbol	Buffer	Meaning
h^*	`_target_apex_height`	commanded target apex height
h_0	`drop_height`	root height latched after reset
$\hat{h}_{\text{apex}}$	`last_apex_height`	latched valid-apex root height
$\hat{h}^*_{\text{apex}}$	`last_apex_target_height`	target active at the latched valid apex
b^{apex}	`is_apex`	one-step valid-apex flag
`armed`	`_apex_armed`	next apex can be counted

5 Observations and Actions

The observation design is split into three reusable blocks: deployable robot observations, privileged base-state observations, and privileged trampoline observations. Different methods combine these blocks differently, but the final deployable actor should use only the deployable block.

Deployable observations. The deployable actor observation is

$$\boldsymbol{o}_t^{\text{dep}} = \left[h_t^*, \boldsymbol{g}_t^b, \boldsymbol{\omega}_t^b, \tilde{\boldsymbol{q}}_t, \tilde{\dot{\boldsymbol{q}}}_t, \boldsymbol{a}_{t-1}\right]. \tag{9}$$

Here h_t^* is the commanded target apex height, $\boldsymbol{g}_t^b$ is projected gravity in the base frame, $\boldsymbol{\omega}_t^b$ is base angular velocity, $\tilde{\boldsymbol{q}}_t$ and $\tilde{\dot{\boldsymbol{q}}}_t$ are relative joint position and joint velocity, and $\boldsymbol{a}_{t-1}$ is the previous action. This block is meant to be hardware-realistic: it does not include world root position, world root orientation, base linear velocity, or true trampoline parameters.

During training, additive noise is applied to projected gravity, base angular velocity, joint position, and joint velocity in the actor observation. The current noise ranges are set in `go2_rebounce_env_cfg.py` and should be updated after real-robot measurement.

Base privileged observations. The base privileged block is

$$\boldsymbol{o}_t^{\text{base}} = \left[\boldsymbol{p}_t^w, \boldsymbol{q}_t^w, \boldsymbol{v}_t^b\right], \tag{10}$$

where $\boldsymbol{p}_t^w$ is root position in the world frame, $\boldsymbol{q}_t^w$ is the unique world root quaternion, and $\boldsymbol{v}_t^b$ is base linear velocity. This block is available to the asymmetric critic and privileged teacher variants, but not to deployable actors.

Trampoline privileged observations. The trampoline privileged block is the normalized true trampoline-parameter vector

$$\boldsymbol{o}_t^{\text{tramp}} = [\log(E/E_0), (m-m_0)/s_m, (\nu-\nu_0)/s_\nu, (\mu_d-\mu_0)/s_\mu, \log(d_e/d_{e,0})]\,. \tag{11}$$

It contains Young's modulus E, trampoline mass m, Poisson ratio ν, dynamic friction μ_d, and elasticity damping d_e. This block is used only for privileged teachers, teacher critics, and future latent-adaptation training signals. It should not be passed directly to a deployable actor at test time.

Observation groups. The standard deployable actor uses only

$$\boldsymbol{o}_t^{\pi} = \boldsymbol{o}_t^{\text{dep}}.$$

The privileged critic appends base state,

$$\boldsymbol{o}_t^{V} = \left[\boldsymbol{o}_t^{\text{dep}}, \boldsymbol{o}_t^{\text{base}}\right],$$

and the privileged teacher appends both base and trampoline information,

$$\boldsymbol{o}_t^{T} = \left[\boldsymbol{o}_t^{\text{dep}}, \boldsymbol{o}_t^{\text{base}}, \boldsymbol{o}_t^{\text{tramp}}\right].$$

Recurrent teacher variants may omit the quaternion depending on the concrete environment class, but they still follow the same separation: deployable terms plus privileged base/trampoline terms.

Observation history. The history environment stacks deployable observations only. It does not stack or expose privileged base or trampoline observations to the actor. With history length H, the actor input is

$$\boldsymbol{o}_t^{\pi,H} = \left[\boldsymbol{o}_{t-H+1}^{\text{dep}}, \ldots, \boldsymbol{o}_t^{\text{dep}}\right], \tag{12}$$

and Isaac Lab flattens the history dimension before passing it to the MLP. The current history length is

$$H = 10.$$

Since the control step is

$$\Delta t = 4 \times 0.005 \text{ s} = 0.02 \text{ s},$$

this corresponds to

$$T_{\text{hist}} = H\Delta t = 10 \times 0.02 = 0.20 \text{ s}.$$

Thus the history MLP receives the most recent 0.20 s of deployable robot observations and previous actions.

Actions. The policy action is a 12-DoF joint-position target with the default Go2 joint offset and action scale from the environment config. In the trampoline variant, a separate non-policy action term continuously repins the trampoline rim; this term does not consume policy actions.

6 DOB Contact Sensor

Both contact-sensor implementations estimate foot contact forces with the same disturbance-observer structure. The difference is not the algebraic estimator, but the dynamics backend and the floating-base velocity convention.

Pinocchio implementation. The Pinocchio version builds a floating-base model from the URDF and uses the Pinocchio free-flyer convention. Its generalized velocity is

$$\boldsymbol{v}^P = \begin{bmatrix} {}^B\boldsymbol{v}_{WB} \\ {}^B\boldsymbol{\omega}_{WB} \\ \dot{\boldsymbol{q}}_j \end{bmatrix}, \qquad \boldsymbol{\tau} = \begin{bmatrix} \mathbf{0}_6 \\ \boldsymbol{\tau}_j \end{bmatrix}. \tag{13}$$

Here the base linear and angular velocities are expressed in the body frame. The measured generalized acceleration is approximated by finite difference:

$$\dot{\boldsymbol{v}}^P_{\text{meas},k} = \frac{\boldsymbol{v}^P_k - \boldsymbol{v}^P_{k-1}}{\Delta t}. \tag{14}$$

Pinocchio computes

$$\boldsymbol{M}^P(\boldsymbol{q}), \qquad \boldsymbol{h}^P(\boldsymbol{q}, \boldsymbol{v}^P) = \boldsymbol{C}^P(\boldsymbol{q}, \boldsymbol{v}^P)\boldsymbol{v}^P + \boldsymbol{g}^P(\boldsymbol{q}). \tag{15}$$

The no-contact, free generalized acceleration is

$$\dot{\boldsymbol{v}}^P_{\text{free}} = \left(\boldsymbol{M}^P(\boldsymbol{q})\right)^{-1} \left(\boldsymbol{\tau} - \boldsymbol{h}^P(\boldsymbol{q}, \boldsymbol{v}^P)\right). \tag{16}$$

The DOB residual is

$$\boldsymbol{r}^P = \boldsymbol{M}^P(\boldsymbol{q}) \left(\dot{\boldsymbol{v}}^P_{\text{meas}} - \dot{\boldsymbol{v}}^P_{\text{free}}\right), \tag{17}$$

or equivalently

$$\boldsymbol{r}^P = \boldsymbol{M}^P(\boldsymbol{q})\dot{\boldsymbol{v}}^P_{\text{meas}} + \boldsymbol{h}^P(\boldsymbol{q}, \boldsymbol{v}^P) - \boldsymbol{\tau}. \tag{18}$$

For foot i, let $\boldsymbol{J}^P_{i,\text{lin}}(\boldsymbol{q}) \in \mathbb{R}^{3\times n_v}$ be the linear foot-position Jacobian in the local-world-aligned frame. With the Go2 joint ordering used by the code, the three-joint block for leg i is

$$\mathcal{B}_i = \{6 + 3(i-1),\ 6 + 3(i-1) + 1,\ 6 + 3(i-1) + 2\}. \tag{19}$$

The estimated contact force is obtained from

$$\left(\boldsymbol{J}^P_{i,\text{lin}}(\boldsymbol{q})[:, \mathcal{B}_i]\right)^T \boldsymbol{f}^P_i = \boldsymbol{r}^P[\mathcal{B}_i], \tag{20}$$

therefore

$$\boldsymbol{f}^P_i = \left(\left(\boldsymbol{J}^P_{i,\text{lin}}(\boldsymbol{q})[:, \mathcal{B}_i]\right)^T\right)^{-1} \boldsymbol{r}^P[\mathcal{B}_i]. \tag{21}$$

GPU / PhysX implementation. The GPU version uses PhysX dynamics tensors directly and keeps the computation batched on the GPU. Its generalized velocity is

$$\boldsymbol{v}^G = \begin{bmatrix} {}^W\boldsymbol{v}_{WB} \\ {}^W\boldsymbol{\omega}_{WB} \\ \dot{\boldsymbol{q}}_j \end{bmatrix}, \qquad \boldsymbol{\tau} = \begin{bmatrix} \mathbf{0}_6 \\ \boldsymbol{\tau}_j \end{bmatrix}. \tag{22}$$

The base velocity is therefore expressed in the world frame instead of the body frame. The measured acceleration is

$$\dot{\boldsymbol{v}}^G_{\text{meas},k} = \frac{\boldsymbol{v}^G_k - \boldsymbol{v}^G_{k-1}}{\Delta t}. \tag{23}$$

PhysX provides

$$\boldsymbol{M}^G(\boldsymbol{q}), \qquad \boldsymbol{h}^G(\boldsymbol{q},\boldsymbol{v}^G) = \boldsymbol{h}^G_{\text{coriolis}}(\boldsymbol{q},\boldsymbol{v}^G) + \boldsymbol{h}^G_{\text{gravity}}(\boldsymbol{q}), \qquad \boldsymbol{J}^G_{i,\text{lin}}(\boldsymbol{q}). \tag{24}$$

The free acceleration and residual are

$$\dot{\boldsymbol{v}}^G_{\text{free}} = \left(\boldsymbol{M}^G(\boldsymbol{q})\right)^{-1}\left(\boldsymbol{\tau} - \boldsymbol{h}^G(\boldsymbol{q},\boldsymbol{v}^G)\right), \tag{25}$$

$$\boldsymbol{r}^G = \boldsymbol{M}^G(\boldsymbol{q})\left(\dot{\boldsymbol{v}}^G_{\text{meas}} - \dot{\boldsymbol{v}}^G_{\text{free}}\right) = \boldsymbol{M}^G(\boldsymbol{q})\dot{\boldsymbol{v}}^G_{\text{meas}} + \boldsymbol{h}^G(\boldsymbol{q},\boldsymbol{v}^G) - \boldsymbol{\tau}. \tag{26}$$

Before the leg blocks are extracted, the PhysX generalized-coordinate ordering is permuted to match the same CSV/Pinocchio joint ordering. After this permutation, the same block $\mathcal{B}_i$ is used:

$$\left(\boldsymbol{J}^G_{i,\text{lin}}(\boldsymbol{q})[:,\mathcal{B}_i]\right)^T \boldsymbol{f}^G_i = \boldsymbol{r}^G[\mathcal{B}_i], \tag{27}$$

$$\boldsymbol{f}^G_i = \left(\left(\boldsymbol{J}^G_{i,\text{lin}}(\boldsymbol{q})[:,\mathcal{B}_i]\right)^T\right)^{-1} \boldsymbol{r}^G[\mathcal{B}_i]. \tag{28}$$

Where the two estimates differ. The final force solve uses only the three joint-space residual rows for each leg. This is why the two estimates have the same leading-order interpretation when joint state, joint torque, and joint ordering match. The remaining difference enters through the floating-base part of the dynamics. In block form,

$$\boldsymbol{r}_j = \boldsymbol{M}_{jb}\left(\dot{\boldsymbol{v}}_{b,\text{meas}} - \dot{\boldsymbol{v}}_{b,\text{free}}\right) + \boldsymbol{M}_{jj}\left(\ddot{\boldsymbol{q}}_{j,\text{meas}} - \ddot{\boldsymbol{q}}_{j,\text{free}}\right). \tag{29}$$

The term with $\boldsymbol{M}_{jb}$ is the base-joint inertia coupling. Since $\boldsymbol{v}^P_b$ is body-frame and $\boldsymbol{v}^G_b$ is world-frame, this coupled term and the corresponding nonlinear dynamics terms are not exactly identical. For moderate base speeds the joint block is usually dominated by the joint motion and torque terms; for aggressive body rotation, impact, or tumbling, the coupling term can make the Pinocchio and GPU estimates visibly diverge.

7 Apex Event and Metrics

A valid apex is detected by a root vertical velocity sign change gated by the apex re-arm state and a geometric foot-clearance condition:

$$b^{\text{vel}}_t = (\dot{z}_{t-1} > 0) \wedge (\dot{z}_t \le 0), \tag{30}$$

$$b^{\text{feet}+}_t = \mathbb{I}\left[\min_{i\in\mathcal{F}} z^{\text{local}}_{i,t} > z_{\text{surface}} + 0.08\right], \tag{31}$$

where $\mathcal{F}$ is the set of four Go2 feet and $z_{\text{surface}} = 0$ in the local support frame. In the implementation this flag is named `feet_above_clearance`.

To avoid counting multiple vertical velocity zero-crossings around the same physical apex, the apex detector is armed only once per bounce. After a valid apex is counted, it is disarmed. It is re-armed only when the feet return below the same clearance threshold:

$$b^{\text{feet}-}_t = \mathbb{I}\left[\max_{i\in\mathcal{F}} z^{\text{local}}_{i,t} \le z_{\text{surface}} + 0.08\right]. \tag{32}$$

In the implementation this re-arm flag is named `feet_below_clearance`. Thus an apex counted by the metric/reward/timeout logic is the valid apex

$$b^{\text{valid}}_t = \texttt{armed}_t \wedge b^{\text{vel}}_t \wedge b^{\text{feet}+}_t. \tag{33}$$

The command term is the owner of this apex state machine. When b^{valid}_t fires, it latches the apex height and the target active at that instant as `last_apex_height` and

`last_apex_target_height`. The target is latched because command resampling can occur after event detection and before the delayed reward reads the event. Reward and termination terms consume this command-owned event one manager step later; this avoids maintaining duplicate apex detectors in reward and termination code.

The logged count metrics are:

Metric	Condition	Interpretation
`apex_count`	b_t^{valid}	valid apex count
`height_matched_apex_count`	$b_t^{\text{valid}} \wedge \lvert h_t - h^* \rvert < 0.05$	apex count close to target height
`error_peak_height`	mean $\lvert h_t - h^* \rvert$ over counted apexes	height tracking error

8 Termination

The current termination terms are:

Term	Condition	Role
`base_orientation`	base tilt angle > 0.6 rad	failure
`non_foot_contact`	non-foot rigid contact > 1 N	failure
`no_valid_apex_timeout`	no valid apex for 2 s	failure
`time_out`	20 s reached	successful long rollout if other failures absent

`no_valid_apex_timeout` reads the command-owned valid-apex pulse and resets its timer whenever that pulse is observed. This prevents a standing policy from simply waiting for the long time limit.

9 Rewards

The weighted reward is

$$\begin{aligned} R_t = \Delta t \Big(& -250\, r_t^{\text{fail}} + 50\, r_t^{\text{apex}} \\ & - 1.5 \times 10^{-2}\, r_t^{\text{energy}} \\ & - 2\, r_t^{\text{orient}} - 0.5\, r_t^{\text{inplace}} - 10^{-2}\, r_t^{\text{action}} - 0.05\, r_t^{\text{joint}} - 10\, r_t^{\text{limit}} \Big). \end{aligned} \tag{34}$$

The failure pulse is

$$r_t^{\text{fail}} = \mathbb{I}[\texttt{base\_orientation}] + \mathbb{I}[\texttt{non\_foot\_contact}] + \mathbb{I}[\texttt{no\_valid\_apex\_timeout}]. \tag{35}$$

There is no separate normal-timeout penalty in the current setup. A rollout that reaches `time_out` without triggering a failure termination is treated as a successful long rollout.

For the delayed apex pulse reward, the height error is computed from the latched command event:

$$e_t^{h,\text{apex}} = \lvert \texttt{last\_apex\_height} - \texttt{last\_apex\_target\_height} \rvert. \tag{36}$$

The flat-body gate is

$$g_t^{\text{flat}} = \exp\left(-\frac{\left\| \boldsymbol{g}_{t,xy}^b \right\|^2}{0.35^2} \right), \tag{37}$$

The height reward does not include an additional soft foot-clearance gate; foot clearance is enforced only by the valid-apex event detector. The apex pulse reward is

$$r_t^{\text{apex}} = \mathbb{I}[b_{t-1}^{\text{valid}}] \exp\left(-\left(\frac{e_t^{h,\text{apex}}}{0.10}\right)^2\right) g_t^{\text{flat}}. \tag{38}$$

The remaining penalties are standard orientation, action-rate, joint-deviation, and joint-position-limit regularizers. The in-place term penalizes drift from the reset xy position and reset yaw:

$$r_t^{\text{inplace}} = \left\| \frac{\boldsymbol{p}_{xy,t} - \boldsymbol{p}_{xy,0}}{0.25} \right\|^2 + \left(\frac{\text{wrap}(\psi_t - \psi_0)}{0.5}\right)^2. \tag{39}$$

Reward scale. In the current 20 s setup, a single perfect apex pulse contributes

$$50 \cdot 0.02/20 = 0.05$$

to the logged `Episode_Reward/rebounce_height`. A single failure termination contributes approximately

$$-250 \cdot 0.02/20 = -0.25$$

for the environments where that failure happened, before averaging over all reset environments. With the current absolute-work weight, a typical sparse energy pulse of $W^{\text{abs}}/h^* \approx 250$ contributes

$$-1.5 \times 10^{-2} \cdot 250 \cdot 0.02/20 \approx -0.00375$$

to the logged `Episode_Reward/energy_penalty` for one valid apex.

10 Implementation Files

File	Purpose
`go2_rebounce_env_cfg.py`	scene, reset, reward, termination config
`mdp/commands.py`	rebounce command, apex metrics, re-arm state, energy metrics
`mdp/events.py`	reset from sampled drop height
`mdp/rewards.py`	height reward, energy penalty, and failure pulses
`mdp/terminations.py`	timeout-without-apex termination
`scripts/rsl_rl/play-rebounce.py`	play script with apex height and energy printout

11 Experiment Roadmap

The experiment order should separate task-definition changes from style, efficiency, and final ablations.

11.1 Step 0: Freeze the Current Continuous-Rebounce Baseline

First train and save a clean baseline with the current config. Record the git state, run ID, checkpoint, and videos. The expected healthy indicators are:

- `Episode_Termination/time_out` close to 1,
- `no_valid_apex_timeout` close to 0,

- `apex_count` close to the visually observed bounce count,
- `height_matched_apex_count`/`apex_count` high,
- `error_peak_height` low.

11.2 Step 1: Decouple Initial Drop Height from Target Apex Height

This has been implemented in the current setup. The target apex height and the initial drop height are sampled independently. The target uses a short hopping-init curriculum; the drop height always uses the full range:

$$h^* = 0.5 \text{ before } 1000 \times 24 \text{ steps, then } h^* \sim \mathcal{U}(0.5, 1.2), \qquad h_{\text{drop}} \sim \mathcal{U}(0.5, 1.2).$$

The height rewards and metrics now compare apex height against h^* rather than against h_{drop}. This tests whether the policy controls rebound height, rather than only recovering to the height from which it was dropped.

For fixed-condition evaluation, the eval script sets $h_{\text{drop}} = h^*$ by default unless `drop_height` is explicitly swept. This removes one nuisance variable when comparing policies across trampoline parameters.

11.3 Step 2: Add In-Place Hopping

This has also been implemented in the current setup with a small dense penalty on displacement from the reset xy position and reset yaw. Suggested diagnostics for the run are

$$\|\boldsymbol{p}_{xy,t} - \boldsymbol{p}_{xy,0}\|, \qquad \|\boldsymbol{v}_{xy,t}\|, \qquad |\psi_t - \psi_0|, \qquad |\dot{\psi}_t|.$$

The current weight is intentionally modest so vertical rebounce quality is not suppressed. The goal is to keep the robot rebounding in place without sacrificing apex-height tracking.

11.4 Step 3: Minimal Leg-Motion Regulation

Before optimizing energy, first remove obvious high-frequency or visually unacceptable leg motion. This should be a light regulation pass, not a strong posture-imitation objective. The goal is to prevent wasted actuator motion and unsafe leg swings while still allowing the robot to shape its body and legs to use the trampoline.

Test the following regularization variants one at a time:

1. increase `joint_deviation_l1` weight, e.g. $-0.05 \rightarrow -0.10$ first; avoid jumping immediately to a strong posture penalty because it can suppress useful trampoline-exploitation strategies;
2. add a small joint-velocity penalty to reduce fast flight leg motion;
3. test whether the command-side hard foot-clearance apex gate is still needed after removing the reward-side soft foot-clearance gate.

This step should be done before energy optimization so that the energy term is not merely compensating for a leg-motion artifact. After each variant, check `apex_count`, `height_matched_apex_count`, `error_peak_height`, in-place drift, and videos. If height tracking or survival degrades, the regularization is too strong.

11.5 Step 4: Add Energy Optimization on a Fixed Trampoline

Only after the behavior is stable and visually acceptable, add energy terms on the fixed trampoline. Keeping trampoline parameters fixed during this step isolates the question: can the policy maintain commanded apex height with less active work by using the trampoline's passive energy storage?

Two mechanical-work definitions should be tested:

$$P_t^+ = \sum_j \max(\tau_{j,t}\dot{q}_{j,t}, 0), \qquad P_t^{\text{abs}} = \sum_j |\tau_{j,t}\dot{q}_{j,t}|.$$

The reward is sparse. Between two valid apex events, the energy command accumulates

$$W_k^+ = \sum_{t\in k} P_t^+ \Delta t, \qquad W_k^{\text{abs}} = \sum_{t\in k} P_t^{\text{abs}} \Delta t.$$

At the next valid apex, the reward penalty is normalized by the commanded target height h_k^*:

$$c_k^{\text{energy}} = \frac{W_k}{\max(h_k^*, \epsilon)}.$$

Using the target height for the reward prevents the policy from jumping higher than requested only to dilute the work-per-height penalty. The reported metrics below still use the achieved apex height, which keeps them interpretable as the actual work needed per meter of realized rebound height.

The implementation exposes this through the `mode` parameter of `energy_penalty` in `go2_rebounce_env_cfg.py`. Set `mode=positive` for W_k^+/h_k^* or `mode=absolute` for W_k^{abs}/h_k^*. The raw reward follows the same convention as the apex-height pulse: Isaac Lab applies the global Δt scaling, and the event scale is handled in the configured weight. With the current $\Delta t = 0.02$ s, the current absolute-work trial weight is

$$w_{\text{energy}} = -1.5 \times 10^{-2}.$$

The current training curriculum has two switches. First, the hopping-init phase keeps the commanded target at $h^* = 0.5$ m and keeps the trampoline fixed:

$$N_{\text{init}} = 1000 \times 24 = 24000$$

manager steps. Before this threshold, resets use

$$h^* = 0.5$$

and the fixed trampoline

$$E = 8.0 \times 10^4, \quad m = 10, \quad \mu_d = 0.8, \quad d_e = 0.02, \quad \nu = 0.35.$$

After this threshold, the target range returns to $h^* \sim \mathcal{U}(0.5, 1.2)$ and reset-time trampoline randomization switches to the configured DR ranges. The drop-height range is unchanged by this curriculum.

Second, the energy reward term starts with zero weight and is enabled later:

$$N_{\text{energy}} = 5000 \times 24 = 120000.$$

This lets the policy first discover sustained rebounding and then adapt to trampoline variation before the sparse work-per-height penalty can make passive non-jumping attractive. The positive-work term penalizes active mechanical energy injected by the motors:

$$\tau_{j,t}\dot{q}_{j,t} > 0.$$

It is the preferred first test because it discourages the robot from actively "kicking" to create apex height while still allowing negative work for landing stabilization. The absolute-work term penalizes both positive and negative mechanical work, so it also discourages active braking:

$$r_t^{\text{abs\_work}} = r_t^{\text{pos\_work}} + \sum_j \max(-\tau_{j,t}\dot{q}_{j,t}, 0).$$

It should be tested after the positive-work variant, or with a much smaller weight, because too much absolute-work penalty can reduce useful damping and postural stabilization during trampoline compression.

A torque-squared term,

$$\sum_j \tau_{j,t}^2,$$

is also useful as a regularizer, but it mainly discourages large torques rather than directly measuring active mechanical work.

The previous positive-work run shows the intended first-order behavior: the logged `Episode_Reward/energy_penalty` is roughly 10% of `Episode_Reward/rebounce_height`, the valid apex count increases, and `positive_work_per_height` decreases. This indicates that the energy term is influencing the policy without dominating the height-tracking task.

Use a small weight or a curriculum that turns on the energy penalty only after the policy already achieves stable rebounce. Keep the height and survival terms dominant. If energy decreases while `apex_count`, `height_matched_apex_count`, `error_peak_height`, or `time_out` degrades, the energy weight is too large or was introduced too early. Compare policies with positive-work-per-height and absolute-work-per-height; use braking ratio as a diagnostic. A low positive-work policy with high braking ratio may simply be absorbing trampoline energy through active braking rather than using the trampoline efficiently.

The energy command term logs:

Metric	Definition
`Metrics/energy/positive_work_per_height`	mean over valid apexes of W_k^+/h_k^{apex}
`Metrics/energy/absolute_work_per_height`	mean over valid apexes of $W_k^{\text{abs}}/h_k^{\text{apex}}$
`Metrics/energy/braking_ratio`	$W_{\text{episode}}^-/\max(W_{\text{episode}}^{\text{abs}}, \epsilon)$

11.6 Step 5: Add Trampoline Parameter Randomization

After the policy can discover rebounding on the fixed trampoline, introduce trampoline randomization. In the current training config, this is done as a curriculum: the first 1000 PPO iterations use the fixed trampoline listed above while the target height is fixed at 0.5 m. Later resets sample the random ranges and restore the full target range. The current Go2 rebounce trampoline setup uses multi-parameter randomization over stiffness, mass, friction, material damping, and Poisson ratio:

$$\log E \sim \mathcal{U}\left(\log(2.0 \times 10^4), \log(8.0 \times 10^4)\right), \qquad m \sim \mathcal{U}(5, 15),$$

$$\mu_d \sim \mathcal{U}(0.4, 1.2), \qquad d_e \sim \mathcal{U}(0.01, 0.1), \qquad \nu \sim \mathcal{U}(0.25, 0.45),$$

with `damping_scale`=1.0 fixed. Young's modulus is sampled log-uniformly because stiffness changes multiplicatively; the other parameters are sampled uniformly in their current engineering ranges. The earlier first-stage setup used

$$E \sim \text{LogUniform}(4.0 \times 10^4, 1.6 \times 10^5), \qquad m = 10,$$

and remains a useful historical baseline. The current range intentionally shifts toward softer trampolines because the old 8.0×10^4 and 1.6×10^5 settings were too similar in height tracking.

The randomizable trampoline parameters should be introduced in stages:

Parameter	Meaning	Suggested use
E / `youngs_modulus`	membrane stiffness; controls soft versus stiff rebound timing	first-stage randomization
m / `mass`	trampoline inertia; changes response speed and energy exchange timing	first-stage randomization
`elasticity_damping` or `damping_scale`	material energy dissipation	second-stage, one damping knob at a time
`dynamic_friction`	tangential foot–trampoline friction, affecting slip and in-place stability	robustness sweep
ν / `poissons_ratio`	deformation mode and volume preservation	late-stage small-range sweep

Keep numerical and sleep parameters fixed during the main physics randomization study: `vertex_velocity_damping`, `sleep_damping`, `sleep_threshold`, `settling_threshold`, solver iteration counts, mesh resolution, `contact_offset`, and `rest_offset`. These parameters can affect stability and numerical dissipation, but they are less direct descriptions of a real trampoline than stiffness, mass, damping, friction, and Poisson ratio. For visual parameter sweeps, the rebounce play script exposes fixed overrides for E, m, `dynamic_friction`, `elasticity_damping`, `damping_scale`, and `poissons_ratio`.

Use the stiffness range as the first-stage domain-randomization test:

$$\log E \sim \mathcal{U}(\log(2.0 \times 10^4), \log(8.0 \times 10^4)).$$

If this range is too difficult, narrow it temporarily, for example to $E/E_0 \in [0.8, 1.25]$, to separate learning instability from adaptation failure. If stiffness randomization is stable, add mass and material damping next, then friction and Poisson ratio. Randomizing too many parameters too early can make it difficult to tell whether a behavior change came from energy optimization, leg regulation, or trampoline dynamics. It can also encourage a more conservative active-jumping strategy that is robust but less clearly trampoline-efficient.

12 Methods

All deployable methods use the same deployable actor observation unless noted otherwise: commanded target height, projected gravity, base angular velocity, joint position, joint velocity, and previous action. The deployable actor does not receive root position, root quaternion, base linear velocity, or true trampoline parameters. The critic may remain privileged during PPO training. The table uses short environment labels; the full task IDs are listed in the environment-variant table above.

Method	Environment	Actor input	Purpose
MLP + DR	`Trampoline`	instantaneous deployable observation	robust baseline
MLP + history	`Trampoline-history`	flattened deployable observation history, $H = 10$	short-horizon temporal inference
RNN	`Trampoline-RNN`	instantaneous deployable observation plus recurrent state	online temporal inference
Privileged teacher	`Trampoline-Teacher`	deployable terms plus root state and true trampoline parameters	upper bound
MLP student	`Trampoline-Student`	deployable history, trained by action distillation	compress teacher behavior
RNN student	`RNN-Student`	instantaneous deployable observation plus recurrent state, trained by action distillation	recurrent deployable student
RMA	future variant	deployable observation plus learned latent dynamics code	explicit adaptation

12.1 MLP + DR

The MLP + DR policy is the deployable baseline. It is trained directly with PPO on the trampoline task using the instantaneous deployable actor observation. The actor is intentionally partially observable: it must handle changes in trampoline stiffness, mass, damping, friction, and Poisson ratio without being told their values.

Training uses the same hopping-init curriculum as the environment: before 1000×24 manager steps, the target height is fixed at 0.5 m and the trampoline is fixed. After that, the target range returns to $[0.5, 1.2]$ m and the trampoline parameters are randomized on reset. The energy penalty remains disabled until 5000×24 manager steps.

12.2 MLP + History

The history baseline keeps the same MLP policy class but replaces the actor input with a flattened history of deployable observations. The current history length is

$$H = 10,$$

which corresponds to 0.2 s at the 0.02 s control step. The critic remains instantaneous and privileged. This method tests whether a short explicit window is enough to infer bounce phase and trampoline response without recurrent state.

12.3 RNN

The RNN policy uses the instantaneous deployable observation at each control step and carries temporal information in its recurrent hidden state. This keeps the per-step actor input compact while allowing the policy to infer trampoline dynamics from recent interaction history. The first recurrent implementation is an LSTM policy.

The RNN is evaluated under the same task, reward, curriculum, and fixed-condition evaluation protocol as the MLP methods. Any gain should therefore come from temporal inference rather than extra privileged information.

12.4 Privileged Teacher

The privileged teacher is a PPO policy trained with an augmented actor observation. In addition to the deployable terms, the teacher receives root state and normalized true trampoline parameters. The trampoline parameter vector contains stiffness, mass, friction, damping, damping scale, and Poisson ratio in the normalized form defined by the observation term.

This teacher is not deployable. Its purpose is to measure an upper bound for what a policy could do if it knew the hidden trampoline dynamics and full root state. It also provides the action source for teacher-student distillation.

12.5 Teacher-Student Distillation

The distillation methods use RSL-RL's `DistillationRunner`. A trained privileged PPO checkpoint is loaded into the teacher network. During distillation, the teacher receives the privileged `teacher` observation group and the student receives the deployable `policy` observation group:

$$\texttt{obs\_groups} = \{\texttt{"policy"} : [\texttt{"policy"}], \texttt{"teacher"} : [\texttt{"teacher"}]\}.$$

The teacher action is computed without gradient,

$$\boldsymbol{a}_t^T = \pi_T(\boldsymbol{o}_t^T),$$

and the student is optimized to match it:

$$\mathcal{L}_{\text{distill}} = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \left\| \mu_S(\boldsymbol{o}_t^S) - \pi_T(\boldsymbol{o}_t^T) \right\|_2^2 .$$

The MLP student uses the same flattened deployable history as the history baseline. The recurrent student uses instantaneous deployable observations and stores temporal information in the recurrent state.

12.6 RMA

RMA is the planned explicit adaptation variant. A privileged training pathway may use true trampoline parameters or a privileged dynamics code, but the final deployable policy should receive only onboard observations and a latent code estimated from recent history. This separates two questions: whether privileged dynamics information is useful, and whether that information can be inferred online without direct access to the true trampoline parameters.

13 Evaluation Metrics

Each method should be evaluated on fixed conditions rather than only on random training rollouts. A condition fixes the target height and trampoline parameters for a batch of independent 20 s episodes. Metrics are computed per condition and then averaged across rollouts.

Metric	Definition	Purpose
`success_rate`	fraction of rollouts that reach timeout without `base_orientation`, `non_foot_contact`, or `no_valid_apex_timeout`	stability
`failure_distribution`	counts of failed-termination reasons	failure diagnosis
`apex_mean`	mean number of valid apexes per rollout	hopping frequency
`matched_mean`	mean number of apexes satisfying $\lvert h_k - h_k^* \rvert < 0.05$ m	strict tracking count
`height_success_rate_mean`	`matched_mean`/`apex_mean`, computed per rollout then averaged	strict tracking rate
`mae_mean`	mean of $\lvert h_k - h_k^* \rvert$ over valid apexes	absolute height error
`rmse_mean`	root mean square of $h_k - h_k^*$ over valid apexes	large-error sensitivity
`bias_mean`	mean signed error $h_k - h_k^*$	under/over jump
`h_over_target_mean`	mean ratio h_k / h_k^*	normalized height
`p90_mean`	90th percentile of $\lvert h_k - h_k^* \rvert$	tail error
`pos_target_mean`	mean positive motor work per commanded target height, W_k^+ / h_k^*	active work
`abs_target_mean`	mean absolute motor work per commanded target height, W_k^{abs} / h_k^*	total work
`braking_ratio_mean`	$W^-_{\text{episode}} / \max(W^{\text{abs}}_{\text{episode}}, \epsilon)$	braking balance
`xy_drift_mean`	rollout xy displacement from the reset position	in-place behavior
`yaw_drift_mean`	final yaw drift from reset yaw	heading drift
`yaw_rms_mean`	RMS yaw error over the rollout	heading stability
`orientation_rms_mean`	RMS roll/pitch or projected-gravity error	posture quality

Height metrics use the command-owned valid apex events. If a rollout has no valid apexes, its height metrics are undefined and should be interpreted together with `success_rate` and `failure_distribution`. Success alone is not sufficient: a policy can survive the full episode while under-jumping every apex, so comparisons should always report both stability and height-tracking metrics.

13.1 Current Fixed-Condition Results

Table 1 summarizes the current fixed-condition CSV evaluations. Conditions where all methods behave similarly are omitted from the detailed table; the comparison focuses on aggregate

tracking quality and on stress cases where the policies separate. The teacher is the privileged PPO policy. MLP+DR is the deployable feed-forward PPO baseline. History-29200 and RNN-28700 are local deployable baselines. Student-H10 is the history-based distillation student, while Student-Base-8900 and Student-RNN-8900 are the current student checkpoints.

Method	Succ.	HSucc.	MAE	h/h^*	HSucc.@1.2	MAE@1.2	HSucc.@hard	MAE@hard
Teacher	0.999	0.977	0.010	0.993	0.932	0.016	0.797	0.033
Student-Base-8900	0.986	0.956	0.016	0.994	0.870	0.027	0.649	0.043
Student-RNN-8900	0.892	0.754	0.051	0.970	0.509	0.114	0.178	0.188
Student-H10	0.751	0.723	0.038	0.954	0.704	0.042	0.362	0.072
MLP+DR	0.977	0.712	0.054	0.972	0.247	0.127	0.000	0.229
RNN-28700	0.836	0.681	0.110	0.912	0.254	0.281	0.000	0.644
History-29200	0.962	0.745	0.114	0.922	0.296	0.308	0.000	0.660

Table 1: Current fixed-condition rebounce evaluation summary. Succ. is episode success rate, HSucc. is strict apex-height success rate, and MAE is the mean absolute apex-height error in meters. The 1.2 columns average only the 1.2 m target-height conditions. The hard columns average the 1.2 m target with damping 0.1 across trampoline stiffnesses.

The aggregate results show that the teacher remains the strongest policy. Student-Base-8900 is the closest deployable policy: it nearly matches the teacher on low and medium targets, but still loses accuracy and contact robustness on the high-target, high-damping cases. MLP+DR is stable but under-jumps the high target. The recurrent policies and History-29200 mainly fail by under-jumping when $h^* = 1.2$ m, especially at high damping.

Method	HS@hi	MAE	Bias	S@lo	HS@lo	Issue
Teacher	0.797	0.033	-0.033	1.000	1.000	mild high-target under-jump
Student-Base-8900	0.649	0.043	-0.043	1.000	0.999	high-damping non-foot contacts
Student-RNN-8900	0.178	0.188	-0.188	0.864	0.833	high-target under-jump and falls
Student-H10	0.362	0.072	-0.072	0.141	0.443	low-target timeout failures
MLP+DR	0.000	0.229	-0.229	1.000	0.985	stable but under-jumps high target
RNN-28700	0.000	0.644	-0.644	0.974	0.942	severe high-target under-jump
History-29200	0.000	0.660	-0.660	1.000	0.997	severe high-target under-jump

Table 2: Diagnostic stress-condition summary. HS@hi, MAE, and Bias average the $h^* = 1.2$ m, damping 0.1 conditions across stiffnesses. S@lo and HS@lo average the $h^* = 0.5$ m, damping 0.1 conditions across stiffnesses. Bias is signed apex-height error in meters.

13.2 TODO: Real-Robot Observation Noise Measurement

Before finalizing sim-to-real observation corruption, measure the observation noise on the physical robot and use the measured ranges to set the policy observation noise. The required measurements are:

- joint position noise from the robot encoder readings;
- joint velocity noise from the robot state estimator or finite differences of encoder readings;
- IMU angular velocity noise for the base angular-velocity observation;
- mocap or state-estimator base position noise;
- mocap or state-estimator base linear-velocity noise.

For each signal, record at least a static standing segment and a representative rebounce or hopping segment. Estimate bias, standard deviation, outliers, latency, and any filtering delay. The resulting noise magnitudes should be used to update the observation corruption in `go2_rebounce_env_cfg.py`, especially for joint position, joint velocity, base angular velocity, base position, and base linear velocity.

13.3 Step 6: Final Ablation Experiments

Run formal ablations after the final task definition is stable. Suggested variants are:

Variant	Change	Main question
Full	final setup	reference
No apex timeout	remove `no_valid_apex_timeout`	is the anti-wait gate needed?
No foot gate	remove the command-side valid-apex foot-clearance gate	does it prevent standing or foot exploits?
No in-place reward	remove xy/yaw terms	does the policy drift?
No regulation	remove flight-leg regulation	does leg motion return?
No energy reward	remove energy term	efficiency versus performance
No trampoline DR	fix trampoline parameters	robustness versus specialization

For every variant, compare success rate, failure distribution, height error, apex count, height-matched apex count, drift metrics, energy, trampoline setting, and videos.

14 Recommended Order

The recommended order is:

1. keep the current deployable-observation MLP+DR baseline as the reference;
2. add observation history to the MLP and evaluate the same fixed trampoline conditions;
3. test RNN policies under the same observation and evaluation protocol;
4. move to RMA only after the history/RNN baselines reveal which trampoline conditions need online adaptation;
5. measure real-robot observation noise and update observation corruption;
6. run final ablations over reward, termination, energy, and trampoline randomization terms.

This order treats MLP+DR as the current baseline, then adds temporal inference in increasing complexity: explicit history, recurrent memory, and finally an RMA-style latent dynamics estimator.